



BITS Pilani
DIGITAL
Excellence made yours

BITS Pilani Digital

Machine Learning Assignment Report: Titanic Survival Prediction

Subject: Machine Learning

Trimester: Trimester 2

Student: Anik Das

Roll Number: 2025em1100026

Assignment objective

Build and compare a complete supervised learning pipeline for Titanic survival prediction, covering preprocessing, baseline models, decision trees, rule extraction, kNN, and ensembles.

Report basis

Dataset used: Titanic Dataset.
This PDF summarizes validated results from the executed notebook and curated report visuals.

1. Introduction, Dataset, and Preprocessing

This report summarizes a complete supervised learning workflow for Titanic survival prediction. The objective was to clean the data, compare multiple classifiers, and balance predictive performance with interpretability.

The class distribution was moderately imbalanced at 549 non-survivors versus 342 survivors, so the analysis emphasized stratified splitting and F1-score rather than relying on accuracy alone.

Property	Value
Rows	891
Columns	12
Target	Survived
Class balance	0: 549, 1: 342
Missing values	Age 177, Cabin 687, Embarked 2
Duplicate rows	0

Attribute type	Columns
Ordinal	Pclass
Nominal	Sex, Embarked
Numeric	Age, Fare, SibSp, Parch

The dataset mixes ordinal, nominal, discrete, and continuous attributes. `Pclass` was treated as ordinal, `Sex` and `Embarked` as nominal, and `Age`, `Fare`, `SibSp`, and `Parch` as numeric features.

Preprocessing rationale

Identifier-like or sparse fields (`PassengerId`, `Name`, `Ticket`, `Cabin`) were excluded because they were non-predictive, high-cardinality, or mostly missing. `Age` used median imputation and `Embarked` used mode imputation so rows could be retained. Outliers were kept because they represented plausible passengers, especially expensive fares, rather than obvious data-entry errors.

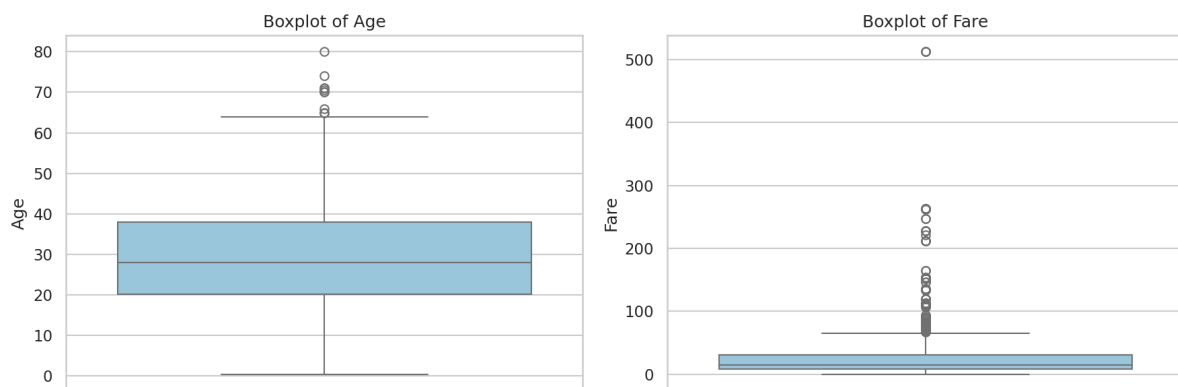


Figure 1. Age and fare boxplots show skew and outliers, especially for `Fare`, which motivated discussion but not deletion.

After preprocessing, the modeling feature set contained `Pclass`, one-hot encoded `Sex` and `Embarked`, and the numeric fields `Age`, `Fare`, `SibSp`, and `Parch`. This retained the strongest interpretable variables while avoiding sparse or identifier-like columns that would add noise without supporting explanation.

2. Modeling Setup and Decision Tree Analysis

Evaluation setup: the data was split using an 80/20 stratified train-test split with `random\_state = 42`. Performance was compared using accuracy, precision, recall, F1-score, and ROC-AUC where available.

Model	Accuracy	F1	ROC-AUC
Logistic Regression	0.8045	0.7244	0.8437
Gaussian Naive Bayes	0.7877	0.7164	0.8191

Decision tree tuning used 5-fold cross-validation on the training data only. The best search result was `max\_depth = None`, `min\_samples\_split = 10`, `min\_samples\_leaf = 1` with cross-validated F1 = 0.7502.

The baseline tree was retained for interpretation because it achieved the stronger held-out result while also keeping the decision paths easy to explain in a classroom setting. That made it a better fit for the final report than the tuned alternative, even though the tuned model won the training-only search.

Top decision-tree feature importance scores are shown below. The values reinforce that survival in this dataset is driven mostly by sex, age, fare, and passenger class, with embarkation contributing much less.

Feature	Importance
Sex_female	0.3150
Age	0.2698
Fare	0.2349
Pclass	0.1097
Embarked_S	0.0227

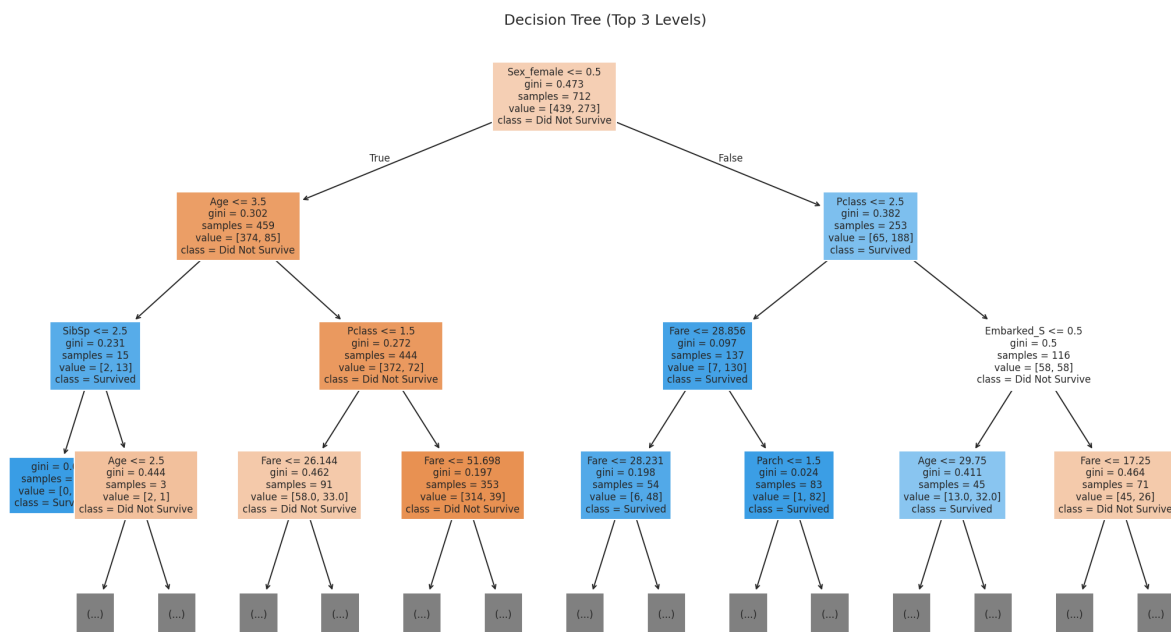


Figure 2. The top levels of the decision tree show that sex, age, fare, and passenger class dominate the early splits.

Decision tree interpretation: the first major splits are driven by sex, then by age, fare, and passenger class. This indicates that a small number of strong predictors capture most of the useful survival structure in the selected feature set.

The structure also highlights useful interaction effects. For example, the meaning of fare changes depending on class and sex, which helps explain why a tree-based model outperformed the purely linear baseline on this dataset.

3. Rule-Based Classification and kNN

A shallower tree was converted into a compact rule-based classifier so that the logic could be presented directly as human-readable conditions rather than only as a plotted tree.

Readable rule set extracted from the shallower tree:

1. Male passenger, age > 3.5, class > 1.5, fare ≤ 51.698 **Prediction:** Did Not Survive. **Support:** 339 passengers. **Purity:** 0.900.
2. Female passenger, class ≤ 2.5, fare > 28.856, parch ≤ 1.5 **Prediction:** Survived. **Support:** 71 passengers. **Purity:** 1.000.
3. Female passenger, class ≤ 2.5, fare ≤ 28.856, age ≤ 37 **Prediction:** Survived. **Support:** 36 passengers. **Purity:** 0.944.
4. Female passenger, class > 2.5, Embarked = S, fare > 17.250 **Prediction:** Did Not Survive. **Support:** 27 passengers. **Purity:** 0.852.
5. Female passenger, class > 2.5, Embarked != S, fare ≤ 8.083 **Prediction:** Survived. **Support:** 23 passengers. **Purity:** 0.870.

These extracted rules capture the most common survival patterns in a directly explainable form. They are especially useful for showing how the model links gender, age, fare, embarkation, and passenger class to the final prediction.

Interpretability tradeoff: the rule-based tree is easier to explain because its logic can be written as a short set of explicit if-then rules. Its held-out performance was weaker than the strongest decision tree, so the gain in interpretability came with a measurable predictive tradeoff.

kNN interpretation: scaling mattered because kNN depends directly on feature distance. Manhattan distance with $k = 9$ performed best, which suggests that a smoother neighborhood definition was more stable for this dataset than more local, noise-sensitive settings.

Larger k values reduce variance by averaging over more neighbors, while smaller k values react more sharply to local variation and therefore behave with lower bias but higher variance.

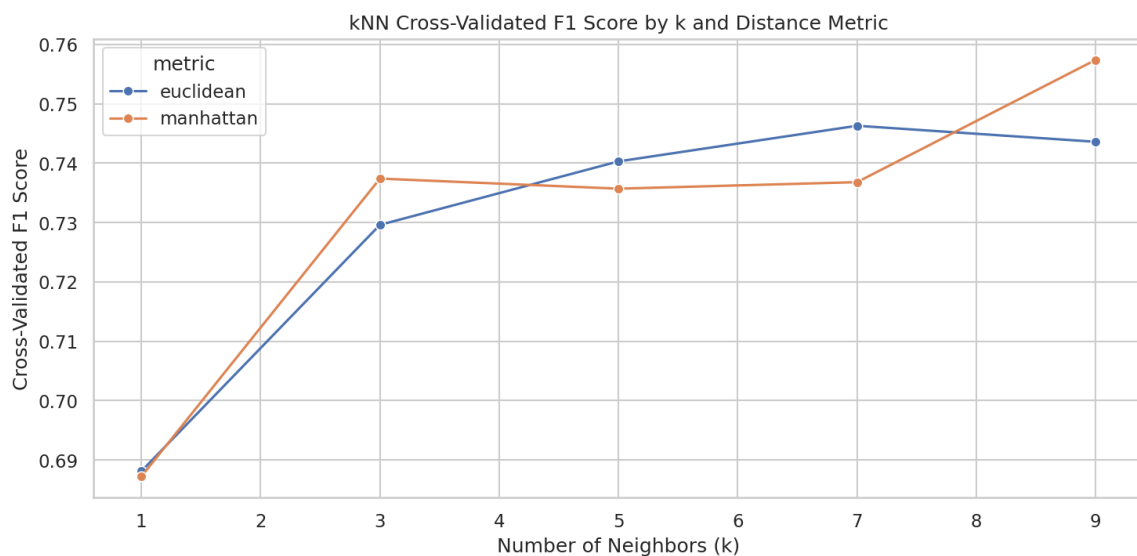


Figure 3. Cross-validated kNN F1-scores improved after scaling, with Manhattan distance at $k = 9$ giving the best overall result.

4. Ensemble Learning and Final Comparison

Three ensemble methods were evaluated: Random Forest, Gradient Boosting, and AdaBoost. In this split they remained competitive but did not clearly outperform the best simpler models. That finding is important because it shows that higher model complexity did not automatically lead to the strongest result.

The final comparison table below summarizes the main held-out metrics used in the report.

Model	Accuracy	F1	ROC-AUC
Decision Tree (Baseline)	0.8268	0.7634	0.8032
Logistic Regression	0.8045	0.7244	0.8437
kNN (k=9, manhattan)	0.7989	0.7188	0.8432
Gaussian Naive Bayes	0.7877	0.7164	0.8191
Random Forest	0.8045	0.7154	0.8395
Gradient Boosting	0.7989	0.7143	0.8181
Decision Tree (Tuned)	0.7821	0.7111	0.8002
AdaBoost	0.7821	0.6977	0.8249
Rule-Based Tree	0.7765	0.6552	0.8144

Ensemble findings

Random Forest, Gradient Boosting, and AdaBoost all produced reasonable results, but none clearly surpassed the strongest simpler models. Random Forest matched Logistic Regression on accuracy but not on F1, while Gradient Boosting and AdaBoost were slightly weaker overall. The main lesson is that additional ensemble complexity did not create enough class-sensitive improvement to justify a more complex final choice on this dataset.

Why simpler models worked well here

The dataset is modest in size, the selected features are low-dimensional, and survival is strongly influenced by a few high-signal variables such as sex, age, fare, and class. Under those conditions, a well-shaped tree and a strong linear baseline can already capture much of the useful structure without very deep or highly aggregated models.

This is an important assignment result because it shows that choosing an interpretable model family did not require sacrificing strong predictive quality. In other words, the simpler models were not just easier to explain; they were also highly competitive on the actual task.

Key comparison insights

- Decision Tree (Baseline) achieved the top F1-score and accuracy, showing that non-linear splits were valuable.
- Logistic Regression remained the strongest non-tree baseline and offered stable overall behavior.
- kNN became competitive only after scaling and cross-validated tuning, confirming the importance of preprocessing.
- Rule-Based Tree gave the clearest explanations, but that interpretability came with lower predictive strength.

Top practical choices

- Decision Tree: best overall predictive choice for this assignment dataset.
- Logistic Regression: strongest compact baseline with stable behavior and solid ROC-AUC.
- Rule-Based Tree: best option when interpretability and stakeholder communication are the main priorities.

Taken together, the final table supports a clear ranking: the baseline Decision Tree is the strongest overall submission model, Logistic Regression is the best compact statistical baseline, and the Rule-Based Tree is the most suitable option for explanation-focused use cases.

5. Final Answers, Risks, and Conclusion

Best-performing model

The baseline Decision Tree was the strongest overall model in this analysis. It achieved accuracy = 0.8268 and F1-score = 0.7634, giving the best balance between overall correctness and class-sensitive performance on the held-out test set. Its strong result is consistent with the way the dataset responds to non-linear interactions among sex, age, fare, and passenger class.

Most interpretable model

The Rule-Based Tree was the most interpretable approach because its logic can be communicated as a short set of explicit if-then rules. Its lower performance is acceptable when transparency and explanation matter more than maximum predictive strength, making it a useful choice for low-risk educational or illustrative settings.

Deployment risks

- The Titanic dataset is small and historical, so generalization is limited.
- Sex and passenger class may encode social or historical bias in the predictions.
- Tree thresholds can create brittle near-boundary decisions for similar passengers.
- Missing or low-quality operational data would reduce reliability and confidence.

Ethical considerations

Even in an educational setting, survival prediction can reflect historical social structure rather than purely individual characteristics. That means apparently strong performance does not remove the need to question fairness, representativeness, and the meaning of the target itself.

Limitations

- Results come from a single train-test split rather than repeated resampling.
- The dataset is too small for strong real-world claims or stable deployment promises.
- Historical survival patterns may not transfer to modern or unrelated settings.
- The chosen feature set deliberately favored clarity over aggressive feature engineering.

Recommendation

If predictive performance is the priority, the baseline Decision Tree is the best choice from this assignment. If explainability is more important than marginal performance gains, the Rule-Based Tree is the better deployment candidate. Logistic Regression remains a useful fallback when a simpler statistical baseline is preferred for comparison or classroom discussion.

Future improvement opportunities

The analysis could be strengthened through repeated cross-validation, probability calibration, more careful threshold selection, and modest feature engineering such as family-size or title-based features. Those extensions might improve robustness, but they were intentionally left out here to keep the workflow transparent and aligned with the assignment scope.

Conclusion

This assignment demonstrated that careful preprocessing, disciplined model comparison, and explicit interpretation can produce a strong supervised learning report. For the Titanic dataset, a relatively simple Decision Tree delivered the best predictive result, while the Rule-Based Tree offered the clearest explanation. The overall comparison therefore highlights a practical tradeoff between accuracy and transparency rather than a single universally best model. That balance is one of the most important lessons from the full workflow.